

Java Offline Pen & Paper Test

Complete Practice Question Bank (145+ Questions)

This booklet is organized to help you prepare for offline / written Java coding tests (the kind used by companies like TCS, Infosys, Accenture, Capgemini, Cognizant, Wipro and others). It covers Numbers & Loops, Arrays, Strings, Patterns, Searching & Sorting, OOP, Collections, Java Streams, Recursion, Exception Handling, and Multithreading — ranging from Easy to Hard, including "trace the output" and concept-explanation style questions that are common in written rounds.

8-Day Practice Plan

Day	Topics	Question Range
Day 1	Numbers & Loops	1 – 20
Day 2	Arrays	21 – 35
Day 3	Strings	36 – 50
Day 4	Patterns & Searching/Sorting	51 – 67
Day 5	Advanced Arrays & Strings	68 – 80
Day 6	OOP Coding	81 – 90
Day 7	Collections & Revision	91 – 123
Day 8	Java Streams	124 – 135
Bonus	Recursion, Exceptions, Threads, Trace Output	136 – 149

How to practice effectively: Hand-write your code on paper before typing it into an IDE — this exposes syntax mistakes (missing semicolons, brackets) that written tests penalize. For "trace the output" questions, write down the value of every variable at each line, not just the final answer. Aim for: Easy in 5–10 min, Medium in 15–20 min, Hard in 25–45 min. After each solution, think of one edge case (empty input, duplicates, negatives).

■ Section 1 — Numbers & Loops (Easy)

1. Reverse a number. Input: 12345 → Output: 54321
2. Check if a number is a Palindrome. Input: 121 → Output: Palindrome
3. Check if a number is an Armstrong number. Example: $153 = 1^3 + 5^3 + 3^3$
4. Print the first N terms of the Fibonacci series.
5. Find the factorial of a number using a loop.
6. Find the factorial of a number using recursion.
7. Check whether a number is prime.
8. Print all prime numbers between 1 and N.
9. Count the number of digits in a number. Input: 12345 → Output: 5
10. Find the sum of digits of a number. Input: 1234 → Output: 10
11. Find the GCD and LCM of two numbers.
12. Swap two numbers without using a third (temporary) variable.
13. Find the largest among three numbers.
14. Print all even and odd numbers from 1 to N.
15. Count even and odd digits in a number. Input: 123456 → Even = 3, Odd = 3
16. Print the multiplication table of a given number.
17. Print the ASCII value of a given character.
18. Check whether a character is a vowel or a consonant.
19. Find a^b (power of a number) without using `Math.pow()`.
20. Convert a decimal number to binary.

■ Section 2 — Arrays (Medium)

21. Find the largest element in an array.
22. Find the smallest element in an array.
23. Find the second largest element in an array without fully sorting it.
24. Reverse an array in place.
25. Find the sum of all elements in an array.
26. Find the average of array elements.
27. Find duplicate elements in an array. Input: 1 2 3 2 5 1 → Output: 1, 2
28. Remove duplicate elements from an array.
29. Count the frequency of every element in an array. Input: 1 2 2 3 3 3 → 1→1, 2→2, 3→3
30. Move all zeros in an array to the end. Input: 1 0 2 0 5 → Output: 1 2 5 0 0
31. Rotate an array to the left by K positions.
32. Rotate an array to the right by K positions.
33. Find the missing number in a sequence. Input: 1 2 4 5 → Output: 3
34. Merge two arrays into one.
35. Check if an array is sorted.

■ Section 3 — Strings (Medium)

- 36. Reverse a string without using a built-in reverse method.
- 37. Check if a string is a palindrome.
- 38. Count the number of vowels and consonants in a string.
- 39. Count the number of words in a sentence. Input: "Java is awesome" → Output: 3
- 40. Remove all white spaces from a string.
- 41. Find duplicate characters in a string. Input: "Programming" → Output: r, g, m
- 42. Count the frequency of each character in a string.
- 43. Check if two strings are anagrams. Example: "listen" and "silent"
- 44. Find the first non-repeated character in a string. Input: "swiss" → Output: w
- 45. Find the first repeated character in a string.
- 46. Toggle the case of every character in a string. "Java" → "jAVA"
- 47. Replace all spaces in a string with '% '.
- 48. Reverse every word in a sentence (keep word order same). "Java Programming" → "avaJ gnimmargorP"
- 49. Find the longest word in a sentence.
- 50. Compress a string using run-length encoding. "aaabbccccc" → "a3b2c4"

■ Section 4 — Star / Number Patterns

Print the following patterns using nested loops (assume 5 rows unless stated otherwise):

51. Print this pattern:

```
*  
**  
***  
****  
*****
```

52. Print this pattern:

```
*****  
****  
***  
**  
*
```

53. Print this pattern:

```
*  
**  
***  
****  
*****
```

54. Print this pattern:

```
1  
12  
123  
1234  
12345
```

55. Print this pattern:

```
5  
54  
543  
5432  
54321
```

56. Print this pattern:

```
A  
AB  
ABC  
ABCD  
ABCDE
```

57. Print this pattern:

```
1  
22  
333  
4444
```

55555

58. Print this pattern:

```
*  
* * *  
* * * * *  
* * * * * * *  
* * * * *  
* * *  
*
```

59. Print this pattern:

```
1  
2 3  
4 5 6  
7 8 9 10
```

60. Print this pattern:

```
A  
BC  
DEF  
GHIJ
```

■ Section 5 — Searching, Sorting & Algorithms (Hard)

61. Two Sum — given an array and a target, return the indices of two numbers that add up to it.
62. Remove duplicates from a sorted array in place.
63. Implement Bubble Sort, showing the array after each pass.
64. Implement Selection Sort.
65. Implement Insertion Sort.
66. Implement Binary Search (iterative and recursive).
67. Implement Linear Search.
68. Find the intersection of two arrays.
69. Find the union of two arrays.
70. Find 'leaders' in an array (an element is a leader if it is greater than all elements to its right). Input: 16 17 4 3 5 2 → Output: 17 5 2
71. Implement Kadane's Algorithm to find the maximum sum subarray.
72. Find the maximum profit from buying and selling a stock once.
73. Find the majority element in an array (appears more than $n/2$ times).
74. Find the longest common prefix among a list of strings. "flower", "flow", "flight" → "fl"
75. Check for valid/balanced parentheses using a Stack. e.g. "()", "{}", "({})"
76. Remove consecutive duplicate characters from a string. "aaabbccddd" → "abcd"
77. Reverse the order of words in a sentence. "I Love Java" → "Java Love I"
78. Find the longest palindromic substring in a string.
79. Print a matrix in spiral order.
80. Multiply two matrices.

■ Section 6 — OOP Coding Questions

These test your understanding of Java syntax and OOP concepts rather than algorithms.

- 81.** Create a Student class with fields, a constructor, getters, and a display() method. Create 2 objects and print their details.
- 82.** Create a BankAccount class with deposit(), withdraw(), and checkBalance() methods.
- 83.** Demonstrate method overloading using an add() method for different parameter types (int, double, three ints).
- 84.** Demonstrate method overriding using Animal and Dog classes.
- 85.** Show runtime polymorphism using a superclass reference pointing to a subclass object.
- 86.** Create an abstract class Shape with subclasses Circle and Rectangle, each implementing an area() method.
- 87.** Implement an interface Printable and create two different classes that implement it.
- 88.** Write a program demonstrating encapsulation using private fields with public getters and setters.
- 89.** Explain and demonstrate constructor chaining using this() and super().
- 90.** Override equals() and hashCode() for an Employee class.

■ Section 7 — Collections Coding Questions

- 91. Count the frequency of each word in a sentence using a HashMap.
- 92. Sort a list of integers using Collections.sort().
- 93. Remove duplicate elements from a list using a HashSet.
- 94. Find the first non-repeating character in a string using LinkedHashMap.
- 95. Sort a list of Employee objects by salary using a custom Comparator.
- 96. Group employees by department using Java Streams.
- 97. Find duplicate numbers in a list using a HashSet.
- 98. Reverse an ArrayList without using Collections.reverse().
- 99. Merge two sorted lists into one sorted list.
- 100. Find the top 3 highest numbers in a list.

■ Section 7B — Collections (Extra / Advanced)

These go beyond basic HashMap/ArrayList usage and are common in mid-level written rounds.

- 101. Sort a HashMap by its values (ascending and descending) and print the sorted result.
- 102. Use a TreeMap to store key-value pairs sorted automatically by key. Print entries in sorted order.
- 103. Use a TreeSet to store a collection of unique elements in natural sorted order.
- 104. Use an Iterator to safely remove elements from a list while looping over it (without a ConcurrentModificationException).
- 105. Implement a min-heap based task scheduler using PriorityQueue, where tasks with lower priority numbers run first.
- 106. Use an ArrayDeque as both a Stack (push/pop) and a Queue (offer/poll) and show the difference in output order.
- 107. Make a custom Employee class implement Comparable so a list of employees can be sorted with Collections.sort() by default (e.g. by id).
- 108. Wrap a List with Collections.unmodifiableList() and demonstrate what happens when you try to modify it.

■ Section 7C — Java Streams

Streams questions are increasingly common in written rounds at product-based and service-based companies alike. Practice writing them without an IDE's autocomplete.

- 109.** Filter only the even numbers from a List using `Stream.filter()` and collect them into a new list.
- 110.** Convert a list of strings to uppercase using `Stream.map()`.
- 111.** Find the sum, average, maximum, and minimum of a list of integers using Streams (`IntStream` / `summaryStatistics`).
- 112.** Sort a list of Employee objects by salary using `Stream.sorted()` with a `Comparator`, without modifying the original list.
- 113.** Count how many times each word appears in a sentence using `Collectors.groupingBy()` and `Collectors.counting()`.
- 114.** Convert a List into a Map (`id` → `name`) using `Collectors.toMap()`.
- 115.** Use `flatMap()` to flatten a `List<List>` into a single List.
- 116.** Remove duplicate elements from a list of integers using `Stream.distinct()`.
- 117.** Use `Stream.reduce()` to calculate the factorial of a number and separately the sum of a list.
- 118.** Filter a list of strings by length greater than 3 and join the results into a single comma-separated string using `Collectors.joining(", ")`.
- 119.** Partition a list of integers into even and odd groups using `Collectors.partitioningBy()`.
- 120.** Generate the first N even numbers using `Stream.iterate()` and print them with `forEach()`.

■ Bonus Section — Recursion, Exceptions, Multithreading & Trace-the-Output

These extra questions dig deeper into concepts frequently used to test true understanding, not just memorized syntax.

- 121.** Solve the Tower of Hanoi problem recursively, and explain its time complexity.
- 122.** Write a recursive program to print all permutations of a string.
- 123.** Implement a singly linked list from scratch (insert, delete, display) without using `java.util.LinkedList`.
- 124.** Detect a cycle in a linked list. Explain the approach you use (e.g. Floyd's cycle detection / slow-fast pointers).
- 125.** Implement a simple Stack using an array, with push, pop, peek, and isEmpty.
- 126.** Write a program using a try-catch-finally block that separately handles `ArithmeticException` and `ArrayIndexOutOfBoundsException`.
- 127.** Create a custom exception class by extending `Exception`, and write a program that throws and catches it.
- 128.** Explain the difference between abstract class and interface, with a short code example of each.
- 129.** Explain the difference between `==` and `.equals()` in Java, with an example.
- 130.** Explain and implement the Singleton design pattern in Java, and discuss how to make it thread-safe.
- 131.** Create two threads (using `Runnable`) that print numbers 1–5 alternately. Explain the role of the `synchronized` keyword.

■ Bonus Section — Multithreading (Extra)

- 132.** Create a thread by extending the `Thread` class and another by implementing `Runnable`. Explain when you would prefer one approach over the other.
- 133.** Demonstrate setting thread priority with `setPriority()` and explain why the JVM does not guarantee execution order based on priority.
- 134.** Write a program that shows a race condition on a shared counter, then fix it using a `synchronized` method. Compare with using a `synchronized` block instead.
- 135.** Implement the classic Producer-Consumer problem using `wait()` and `notify()` on a shared buffer.
- 136.** Use an `ExecutorService` (`Executors.newFixedThreadPool()`) to run five tasks concurrently, then shut the pool down cleanly.
- 137.** Implement a thread-safe counter using `AtomicInteger` and compare it to using a plain `int` with `synchronized`.
- 138.** Write two threads that intentionally deadlock by acquiring two locks in reverse order, then explain how you would redesign the code to avoid the deadlock.

- 139.** Use `thread.join()` so that the main thread waits for two worker threads to finish before printing a final result.
- 140.** Explain the difference between `Thread.sleep()` and `Object.wait()`, with a short example of each.
- 141.** Use a `Callable` and a `Future` to run a task on a thread and retrieve its return value once complete.

■ Bonus Section — Trace-the-Output & Final Program

142. Trace the output of the following program and explain each step:

```
int x = 5;
int y = x++ + ++x;
System.out.println(x + " " + y);
```

143. Trace the output of the following program and explain why this output occurs (runtime polymorphism):

```
class A {
    void show() { System.out.println("A"); }
}
class B extends A {
    void show() { System.out.println("B"); }
}
public class Main {
    public static void main(String[] args) {
        A obj = new B();
        obj.show();
    }
}
```

144. Trace the output of the following program (tests static vs. instance understanding):

```
class Counter {
    static int count = 0;
    Counter() { count++; }
}
public class Main {
    public static void main(String[] args) {
        new Counter(); new Counter(); new Counter();
        System.out.println(Counter.count);
    }
}
```

145. Trace the output of the following program and explain why the volatile keyword matters here (tests visibility of variables across threads):

```
class Flag {
    volatile boolean running = true;
}
public class Main {
    public static void main(String[] args) throws InterruptedException {
        Flag f = new Flag();
        Thread t = new Thread(() -> {
            while (f.running) { /* busy wait */ }
            System.out.println("Thread stopped");
        });
        t.start();
        Thread.sleep(100);
        f.running = false;
    }
}
```

```
}
```

146. Write a program that sorts an array using Bubble Sort and prints the array's state after each pass (combines sorting logic with trace-style output).

End of question bank — 146 questions covering Numbers, Arrays, Strings, Patterns, Algorithms, OOP, Collections, Java Streams, Recursion, Exceptions, Multithreading, and trace-the-output.